BACKEND SAMPLE | INTERVIEW COORDINATION | SYNTHETIC DATA

Retries without a second interview

Jeronimo Munoz Larreta | 9 October 2026 | Independent technical example

Problem: repeated events and lost acknowledgements

A vacancy event arrives twice. A provider can accept a scheduling command and still time out before acknowledging it. Treating a retry as a fresh request can duplicate the appointment. Treating a timeout as success loses the evidence needed to reconcile it.

Injected event or failure	Observed local behavior
Same key + identical payload	One persisted outbox command; duplicate acknowledged
Same key + different slot	409-equivalent conflict; original state preserved
Timeout before provider acceptance	retry + attempt log; zero provider schedules
Lost acknowledgement after acceptance	retry; same durable token; one schedule after retry
Cancel after a pending reschedule	Older command superseded; late revision stays stale

Small mechanism, observable state

Python + SQLite store the payload hash, desired revision, outbox and attempt history. The mock provider persists idempotency tokens and applies only newer revisions. Confirmation is recorded after acknowledgement, not inferred from an attempted call.

```
timeout -> retry -> acknowledgement -> delivered
cancel revision 3 -> old revision 2 superseded
late revision 2 -> stale; final active schedules: 0
```

Reproduce locally: no external services

```
python -B -m unittest -v
python -B demo.py
```

Tests include persisted deduplication across reopening the database, payload conflicts, timeout on either side of acceptance, audited retries, rescheduling and cancellation. All identifiers and timestamps are synthetic.

Scope

One serialized local worker. The provider is a mock with durable tokens and revision-aware updates; a real calendar needs equivalent support or explicit reconciliation. Cancellation is asynchronous. No distributed exactly-once guarantee, real calendar integration, candidate scoring or production deployment is claimed.